Good

Evening

Everyone

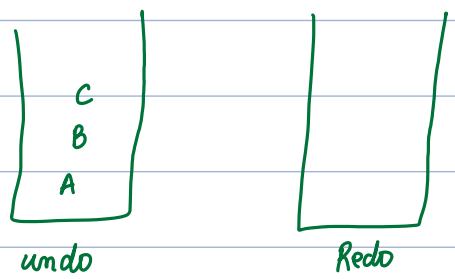
Starting at 9:05 pm

Introduction to Stacks

A stack is a linear data structure that stores information in a sequence, from bottom to top

LIFO - Last in First Out.

A B C



Quiz!.

Operations on stack.

① Push

insert the data on the top of stack
push (data).

② Pop

Pop operation remove one element from top of stack

③ Peek | Top

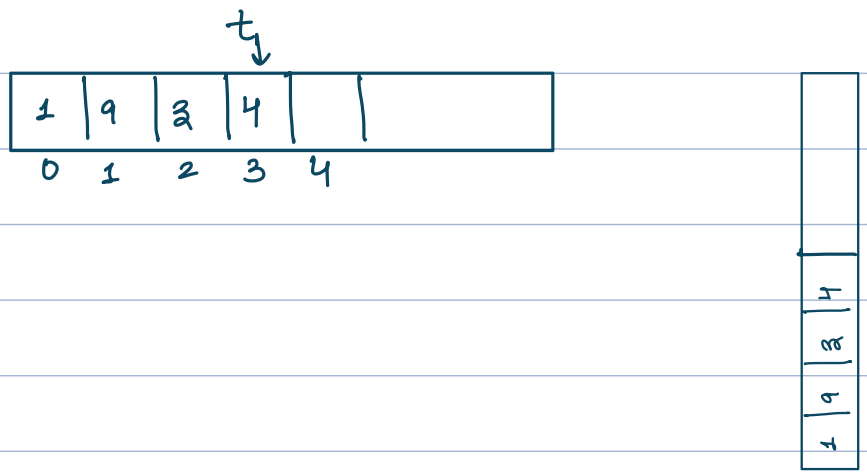
data = peek() - access the data from top of stack without removing it.

④ isEmpty()

↳ gives you True/False if your stack is empty or not

Quiz 2.

PROBLEM 1 : IMPLEMENTATION OF STACKS with ARRAYS



PSEUDOCODE

// Consider array A

A[] t = -1

push operation

```
function push (data) {
    t++
    A[t] = data
}
```

Top / Peek operation

```
function top () {
```

```
    return A[t]
}
```

Pop function

```
function pop () {
    t--
}
```

isEmpty function

```
function isEmpty() {
    return t == -1
}
```

Quiz 3.

What happens when you try to pop an element from an empty stack

OVERFLOW AND UNDER FLOW

Overflow occurs when try to store new data even when there is no empty space.

Resolution

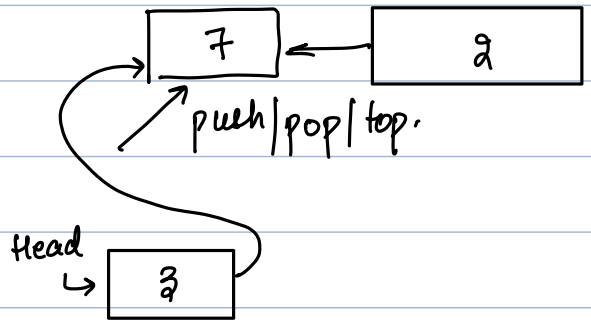
```
function push(x) {
    if (t >= A.size()) {
        return
    }
    t++
    A[t] = x
}
```

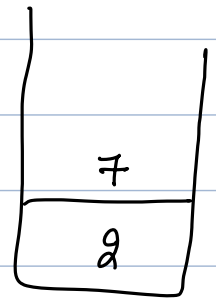
Underflow means when try to perform pop operation or try to access the element.
Again we need to introduce conditions before pop.

```
function pop() {
    if (isEmpty()) {
        return
    }
    t--
}
```

```
function top() {
    if (isEmpty()) -> return -1
    return A[t]
}
```

PROBLEM 2 : IMPLEMENTATION OF STACK USING LINKED LIST





Pseudocode

```
function push (data {
    new_node = Create a new Node with 'data'
    new_node.next = head
    head = new_node

    t++
}

function pop() {
    if ( isEmpty() )    return

    head = head.next
    t--
}
```

```
function top () {
    if (isEmpty()) return -1

    return head.data
}
```

PROBLEM 3 : BALANCED PARANTHESIS CONCEPT with IMPLEMENTATION

Check whether the given sequence is valid ?

{ ( ) } { } ( ) ([ ])

{) (}

Quiz 4. which one of following is balanced

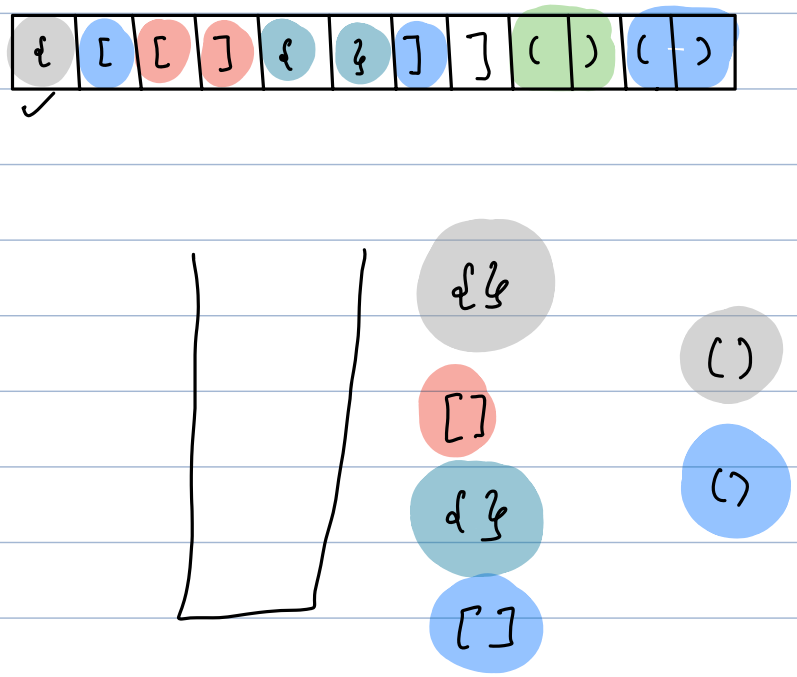
(a+b) * c

(a+b)) * c

(a+b) (c

(a+b) c)

APPROACH



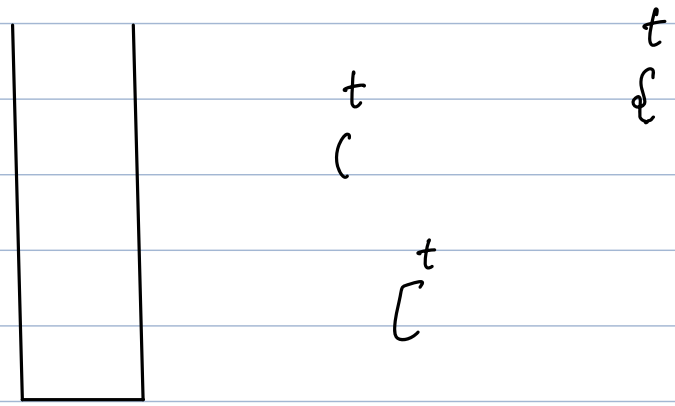
PSEUDOCODE

```
function is_validparenthesis (Sequence) {

    // Initialise stack empty.

    for each character in sequence

        If 'char' is an opening parenthesis '{' or '[' or '('
            Push into the stack

        Else if 'char' is an closing parenthesis '}' or ']' OR ')'

            If the stack is empty :
                Return false

            Pop the top element from the stack into 't'

            If 't' is not matching with the opening
            type of 'char'

            Return false

    }
    if (the stack is not empty) return false

    return true
}
```

DRY RUN

char

{ ( ) [ ] }



return true.

PROBLEM 4 : Remove equal pair of consecutive elements till possible

Given a string, remove equal pair of consecutive elements till possible

Eg → abcddc

⇒ abcc

⇒ ab

Quiz 3.

Eg → a bb c bb c a c x

a c bb c a c x

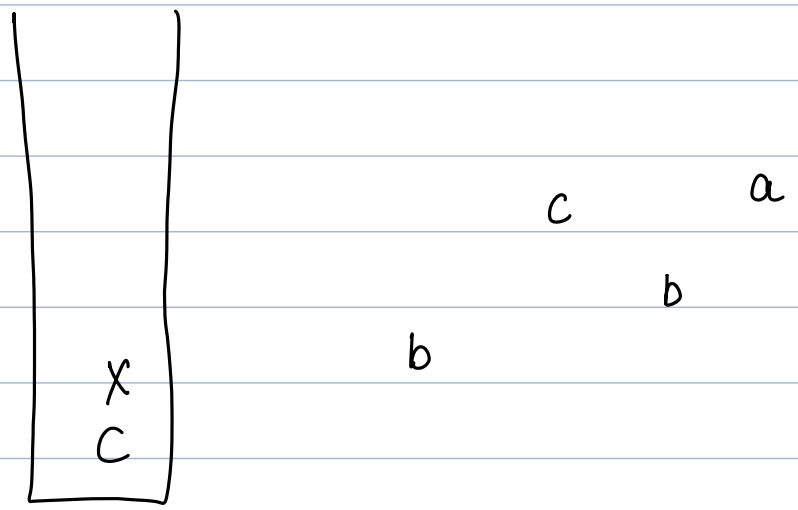
a cc a c x

a a c x

c x

APPROACH

a bb c bbc a c x



result = x

result = top + result.

TC: O(n)

PSEUDOCODE

```
function removeEqualPair (s)

    // Initialize empty array.

    for each character 'char' in s

        If stack is not empty and char matches the element on top of stack.
            Pop the element
        Else
            Push the char into stack
```

}

Initialize a string result.

while the stack is not empty.

Pop and element from stack prepend to result

}

return result

}

PROBLEM 5 : Evaluate Postfix Expression

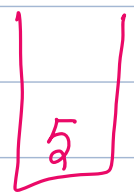
Given a postfix expression, evaluate it

Postfix Expression contains operator after the operands

2+2 ⇒ 2 3 +

Idea :

Example 1 ⇒ [2, 3, +]



3 + 2 = 5

Example 2 [4 3 3 * + 2 -]

2 -
13

= 11

9 +
4 = 13

3 *
3 = 9

Quiz 6.

Evaluate [5 2 * 3 -]

$3 -$
$10 = \underline{7}.$

2
$5 * = 10$

$\boxed{7} \Rightarrow \underline{7}.$

Quiz 7.

Evaluate : [3 5 + 2 - 2 5 * -]

5
$2 * =$
2

5
$3 + = 8$

2
$8 - = 6$

$10 -$
6
$=$
$\Rightarrow 6 - 10 \Rightarrow$
-4

$\boxed{\underline{-4}} \Rightarrow \underline{-4}.$

PSEUDOCODE

TC : $O(n)$

SC : $O(n)$.

```
function evaluate_postfix (expression) {

    // Initialise empty stack.

    for each element 'element' in expression :
        {
            If element is an operand :

                Push 'element' into stack.

            Else if 'element' is operator.

                Pop 'operand 2' from stack.
                Pop 'operand 1' from stack.

                Perform operation 'operand 1' and 'operand 2'

                Push result back on stack
        }

    The final result must be on top of stack.

    Pop and return

}
```

Summary